

EduStack

Your Ultimate Computer Science Hub

PART 2 — ML/AI, Payments, Frontend & Cloud Ops

Complete Interview Preparation Guide

ML/AI | Frontend | Payments | Cloud | DSA Sync | Deployment | Q&As

Developer:	Shubham Kumar CSE Student NIT Patna
Project:	EduStack - CS Resource Hub for Engineering Students
AI Stack:	Python FastAPI + Google Gemini + RAG Engine + pypdf
Payment:	Razorpay Gateway - HMAC-SHA256 Server-Side Verification
Frontend:	Vanilla JS + TailwindCSS + partials.js Micro-Framework
Cloud:	Cloudinary CDN + Nodemailer + MongoDB Atlas + Render.com
Part 2 Covers:	ML/AI, Frontend, Payment, Cloud, DSA Sync, Deployment, 40 Q&As

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub
For Interview Preparation Only — Part 2 of 3

Table of Contents — Part 2

- 11. Razorpay Payment Gateway - Complete Integration**
Order creation, checkout UI, HMAC-SHA256 verification, fraud prevention
- 12. ML / AI Microservice - Python FastAPI**
RAG engine, Gemini API, PDF processing, model fallback strategy
- 13. Frontend Architecture - partials.js System**
Nav injection, auth guards, theme engine, toast system, Razorpay modal
- 14. API Design Patterns & REST Principles**
REST conventions, response format, validation, HTTP status codes
- 15. Cloud Services - File Uploads & Email**
Multer + Cloudinary pipeline, Nodemailer OTP, graceful degradation
- 16. DSA Sheet - Live Google Sheet Sync**
3-layer cache, custom CSV parser, merge strategy, client-side features
- 17. Notification System - Broadcast Design**
readBy array pattern, \$addToSet, badge count, mark-all-read bulk update
- 18. Deployment - Render.com + Atlas**
Environment config, trust proxy, graceful shutdown, production checklist
- 19. Responsiveness & UI Design System**
Breakpoints, dark mode, theme engine CSS vars, micro-animations
- 20. Interview Q&A - Part 2 (40 Questions)**
ML/AI, payments, frontend, cloud, scaling, and project-specific Q&As

SECTION 11

Razorpay Payment Gateway

Complete 3-step flow, HMAC-SHA256 verification, fraud prevention, and code walkthrough

11.1 Why Razorpay & 3-Step Payment Lifecycle

Razorpay is integrated using a 3-step flow that guarantees zero payment fraud. Server creates an order ID, Razorpay opens checkout, and server verifies HMAC signature before setting `isPremium` to true.

FLOW DIAGRAM: Razorpay Secure Payment Lifecycle

Step 1: Step 1: Client sends POST `/api/payments/create-order` -> Server calls Razorpay API -> Payment doc saved as "created"



Step 2: Step 2: Frontend receives `orderId` & opens Razorpay Hosted Checkout UI -> User enters UPI/Card -> Razorpay signs callback



Step 3: Step 3: Client sends signature to POST `/api/payments/verify` -> Server verifies HMAC-SHA256 -> `User.isPremium = true`

Code / Implementation Snippet

```
// server/services/razorpayService.js
const verifyPaymentSignature = (orderId, paymentId, signature) => {
  const message = `${orderId}|${paymentId}`;
  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(message)
    .digest('hex');

  try {
    return crypto.timingSafeEqual(
      Buffer.from(expectedSignature, 'hex'),
      Buffer.from(signature, 'hex')
    );
  } catch {
    return false;
  }
};
```

ML / AI Microservice - Python FastAPI

RAG engine, Gemini API, PDF processing, model fallback strategy

12.1 RAG Engine & FastAPI Code

The RAG engine retrieves relevant CS chunks before prompting Gemini to answer questions accurately.

Code / Implementation Snippet

```
# ml_services/main.py
@app.post('/api/rag/ask')
def ask_ai_tutor(req: RAGQueryRequest):
    relevant_chunks = rag_store.retrieve(req.question, subject=req.subject, top_k=3)
    context_text = '\n--\n'.join([c.content for c in relevant_chunks])
    prompt = f'Context:\n{context_text}\n\nQuestion: {req.question}'

    for model_name in get_available_gemini_models():
        try:
            model = genai.GenerativeModel(model_name)
            response = model.generate_content(prompt)
            if response and response.text:
                return RAGQueryResponse(answer=response.text, sources=relevant_chunks)
        except Exception:
            continue
    raise HTTPException(503, 'AI service unavailable.')
```

Frontend Architecture - partials.js System

Nav injection, auth guards, theme engine, toast system

13.1 window.requireAuth Guard Pattern

Front-end auth guard pattern protects interactive links by validating auth status before executing actions.

Code / Implementation Snippet

```
// client/assets/js/partials.js
window.requireAuth = function(action, label) {
  if (window.currentUser) {
    action();
    return;
  }
  fetch('/api/auth/me', { credentials: 'include' })
    .then(r => r.json())
    .then(d => {
      if (d && d.success && d.data && d.data.user) {
        window.currentUser = d.data.user;
        action();
      } else {
        showAuthModal(label || 'access this feature');
      }
    });
};
```

API Design Patterns & REST Principles

REST conventions, response envelope, validation pipeline

All API endpoints return a predictable JSON response structure containing success, message, and data fields.

Code / Implementation Snippet

```
const sendSuccess = (res, message, data = {}, statusCode = 200) =>
  res.status(statusCode).json({ success: true, message, data });

const sendError = (res, message, statusCode = 400, errors = []) =>
  res.status(statusCode).json({ success: false, message, errors });
```

Cloud Services - File Uploads & Email

Multer memoryStorage + Cloudinary CDN zero-disk pipeline

Multer stores files in RAM as a Buffer, which is streamed directly to Cloudinary CDN without touching server disk.

Code / Implementation Snippet

```
const uploadToCloudinary = (buffer, folder) =>
  new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_stream({ folder }, (err, res) => {
      if (err) return reject(err);
      resolve(res);
    });
    stream.end(buffer);
  });
```

DSA Sheet - Live Google Sheet Sync

3-layer caching strategy and custom quotation-aware CSV parser

The DSA sheet uses a 3-layer fallback system: (1) in-memory 5-minute TTL cache, (2) Live Google Sheet CSV fetch, (3) Disk JSON fallback.

Notification System - Broadcast Design

readBy array pattern and bulk \$addToSet updates

Notifications use an embedded readBy array of user ObjectIds updated via MongoDB \$addToSet operator.

Deployment - Render.com + Atlas

Environment config, trust proxy, and graceful shutdown

Render.com deployment requires `app.set("trust proxy", 1)` so Express correctly reads the client IP from X-Forwarded-For headers.

SECTION 19

Responsiveness & UI Design System

Breakpoints, theme engine CSS vars, and round profile pills

The UI uses CSS custom variables on :root to switch brand colors dynamically across 36 light and dark theme combinations.

Interview Q&A - Part 2

40 Detailed Questions & Answers on Payments, ML, Cloud & Ops

Q: Explain how RAG works in your AI Tutor feature.

Answer:

RAG retrieves relevant course chunks from the knowledge base using keyword intersection scoring before sending the prompt to Gemini. This grounds the answer in actual course material and prevents LLM hallucinations.

- > Retrieval: finds top 3 relevant document chunks.
- > Generation: Gemini generates a factual response using the provided context.

Q: How does Multer memoryStorage prevent ephemeral storage issues on Render?

Answer:

Multer memoryStorage keeps uploaded files as Buffer objects in RAM for the request duration, then streams directly to Cloudinary via `upload_stream()`. Files never touch the server disk, making it safe for Render.com ephemeral containers.

- > Zero disk writes: files live in memory.
 - > Cloudinary `secure_url` stored in MongoDB.
-

